

ONLINE SHOPPING CUSTOMER PURCHASE PREDICTION

Machine Learning Classification Project Report

Objective

To predict whether an online visitor will make a purchase during a browsing session using machine learning classification techniques.

Models: Decision Tree Classifier and Support Vector Machine (SVM)

Dataset: Online Shoppers Intention Dataset

Report basis: Uploaded Jupyter Notebook PDF

Academic Project Report

ABSTRACT

This project develops a machine learning solution for predicting whether an online shopping visitor will complete a purchase during a browsing session. The uploaded notebook uses the Online Shoppers Intention dataset and implements data inspection, cleaning, exploratory data analysis, feature engineering, preprocessing, Decision Tree classification and Support Vector Machine classification. The models are evaluated using accuracy, precision, recall, F1-score and confusion matrices.

The notebook begins with 12,330 records and 18 columns. After checking for missing values and removing 125 duplicate rows, 12,205 records remain. The target variable, Revenue, contains 10,297 non-purchase sessions and 1,908 purchase sessions. The final experiments use an 80:20 stratified train-test split.

The Decision Tree achieves 89.76% accuracy and a 65.37% F1-score, while the SVM achieves 89.43% accuracy and a 59.69% F1-score. The Decision Tree is therefore the stronger overall model in this experiment. Its feature-importance analysis identifies PageValues as the dominant feature, followed by BounceRates, November activity, Total_Duration and ProductRelated_Duration.

1. INTRODUCTION

Online retailers collect behavioral information from website sessions. Such information can be used to understand how visitors browse and to predict whether a session will result in a purchase. In this project, purchase conversion is treated as a binary classification problem with Revenue as the target variable.

The notebook explicitly defines the objective as predicting whether an online visitor will make a purchase during a browsing session using machine learning classification techniques. ■filecite■turn0file0■L3-L6■

1.1 Project Objectives

- Clean and preprocess the web-session data.
- Conduct EDA to understand browsing patterns associated with purchases.
- Create session-level features for duration and browsing activity.
- Build Decision Tree and SVM classification models.
- Evaluate models using accuracy, precision, recall, F1-score and confusion matrices.
- Visualize conversion patterns across traffic, session and platform variables.
- Identify important factors associated with purchase prediction and recommend sales strategies.

1.2 Business Problem

An online retailer wants to predict whether a visitor will purchase during a browsing session so that personalized offers and conversion-improvement strategies can be targeted toward potential buyers.

2. DATASET DESCRIPTION

The notebook loads `online_shoppers_intention.csv`. The dataset initially contains 12,330 rows and 18 columns. The columns include page counts, page durations, bounce rate, exit rate, page value, special-day information, month, operating system, browser, region, traffic type, visitor type, weekend and Revenue.

The notebook confirms the 18 variables and their data types, including 7 float, 7 integer, 2 object and 2 Boolean columns.

filecite turn0file0 L50-L75

Group	Variables
Browsing	Administrative, Informational, ProductRelated
Duration	Administrative_Duration, Informational_Duration, ProductRelated_Duration
Behavior	BounceRates, ExitRates, PageValues, SpecialDay
Context	Month, OperatingSystems, Browser, Region, TrafficType
Visitor	VisitorType, Weekend
Target	Revenue

3. DATA CLEANING AND PREPROCESSING

3.1 Missing Values

The notebook checks every column and reports a total of zero missing values. Therefore, no missing-value imputation is required.

3.2 Duplicate Removal

The notebook identifies 125 duplicate rows and removes them. The dataset changes from $12,330 \times 18$ to $12,205 \times 18$.

3.3 Target Variable

Revenue is converted from Boolean to integer. After duplicate removal, Revenue=False occurs 10,297 times (84.37%) and Revenue=True occurs 1,908 times (15.63%). This demonstrates a clear class imbalance toward non-purchase sessions.

3.4 Feature Engineering

Total_Duration is created as the sum of Administrative_Duration, Informational_Duration and ProductRelated_Duration. Total_Pages is created as the sum of Administrative, Informational and ProductRelated page counts.

3.5 Preprocessing

Month, VisitorType and Weekend are treated as categorical variables. Numerical variables are standardized using StandardScaler, while categorical variables are transformed with OneHotEncoder(handle_unknown='ignore'). The transformations are placed inside a ColumnTransformer and model Pipeline.

4. EXPLORATORY DATA ANALYSIS

EDA is used to examine conversion patterns across traffic source, visitor type, session duration, browsing activity, operating system, bounce rate, exit rate and page value.

4.1 Purchase Distribution

The cleaned dataset contains 1,908 purchase sessions and 10,297 non-purchase sessions. Purchases therefore account for 15.63% of the cleaned observations.



Figure 1. Purchase conversion distribution from the notebook.

4.2 Traffic Source

The notebook calculates mean Revenue by TrafficType. TrafficType 16 has the highest observed conversion rate (0.3333), followed by TrafficType 7 (0.3000) and TrafficType 8 (0.2770). TrafficTypes 12, 15, 17 and 18 have a conversion rate of 0 in this dataset.

```

In [ ]: traffic type

In [16]: traffic_conversion = (
          df.groupby("TrafficType")["Revenue"]
            .mean()
            .sort_values(ascending=False)
          )

          print(traffic_conversion)

TrafficType
16    0.333333
7     0.300000
8     0.276968
20    0.259067
2     0.216569
5     0.215385
10    0.200000
11    0.190283
4     0.154784
14    0.153846
6     0.119639
1     0.109715
9     0.097561
3     0.089419
13    0.059066
19    0.058824
12    0.000000
15    0.000000
17    0.000000
18    0.000000
Name: Revenue, dtype: float64

In [17]: plt.figure(figsize=(10,5))

          traffic_conversion.plot(kind="bar")

          plt.title("Conversion Rate by Traffic Type")
          plt.xlabel("Traffic Type")
          plt.ylabel("Conversion Rate")

          plt.xticks(rotation=0)
          plt.show()

```

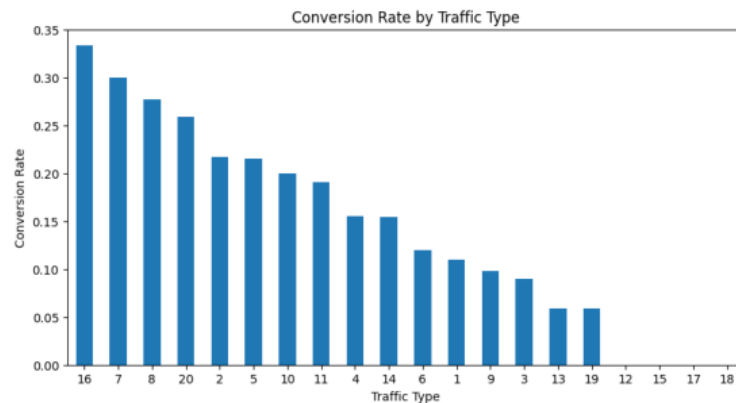
Figure 2. Conversion rate by traffic type.

4.3 Visitor Type

Observed conversion rates are 24.93% for New_Visitor, 19.75% for Other and 14.09% for Returning_Visitor. The categories therefore show different conversion proportions in this dataset.

8/14/26, 4:05 PM

Untitled



```
In [18]: visitor_conversion = (  
    df.groupby("VisitorType")["Revenue"]  
    .mean()  
    .sort_values(ascending=False)  
    )  
  
print(visitor_conversion)
```

```
VisitorType  
New_Visitor      0.249262  
Other             0.197531  
Returning_Visitor 0.140926  
Name: Revenue, dtype: float64
```

```
In [19]: plt.figure(figsize=(8,5))  
  
sns.barplot(  
    data=df,  
    x="VisitorType",  
    y="Revenue"  
    )  
  
plt.title("Conversion Rate by Visitor Type")  
plt.xlabel("Visitor Type")  
plt.ylabel("Conversion Rate")  
  
plt.show()
```

file:///C:/Users/Ashish/Downloads/Untitled.html

6/23

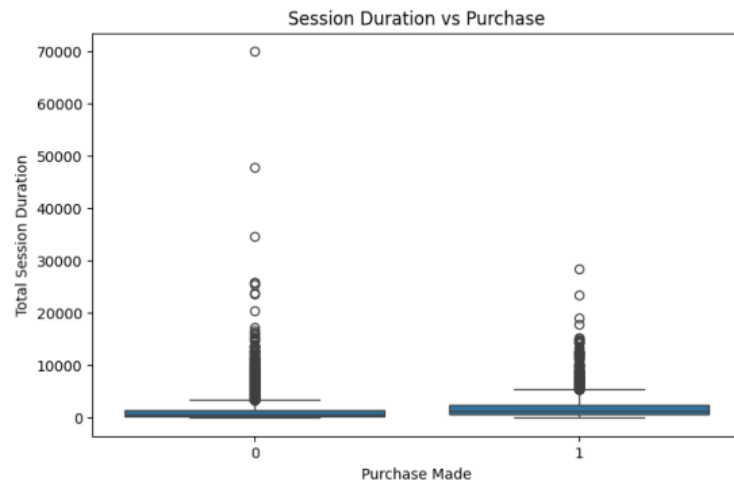
Figure 3. Conversion rate by visitor type.

4.4 Session Duration and Browsing Activity

The notebook creates Total_Duration and Total_Pages and uses boxplots/histograms to compare these variables across purchase outcomes.


```
plt.xlabel("Purchase Made")
plt.ylabel("Total Session Duration")

plt.show()
```



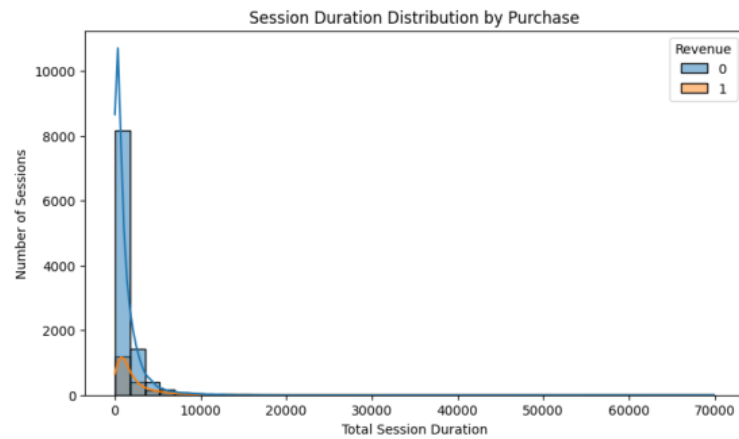
```
In [22]: plt.figure(figsize=(9,5))

sns.histplot(
    data=df,
    x="Total_Duration",
    hue="Revenue",
    kde=True,
    bins=40
)

plt.title("Session Duration Distribution by Purchase")
plt.xlabel("Total Session Duration")
plt.ylabel("Number of Sessions")

plt.show()
```

Figure 4. Session-duration distribution by purchase outcome.



```
In [23]: df["Total_Pages"] = (
    df["Administrative"]
    + df["Informational"]
    + df["ProductRelated"]
)

df[[
    "Administrative",
    "Informational",
    "ProductRelated",
    "Total_Pages"
]].head()
```

```
Out[23]:
```

	Administrative	Informational	ProductRelated	Total_Pages
0	0	0	1	1
1	0	0	2	2
2	0	0	1	1
3	0	0	2	2
4	0	0	10	10

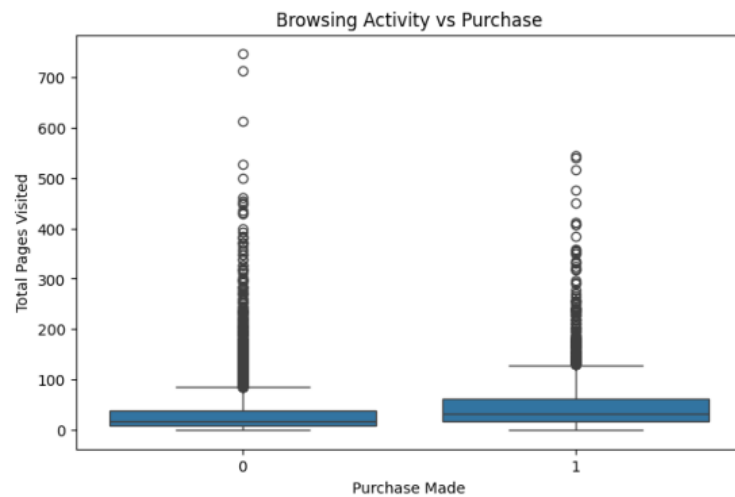
```
In [24]: plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="Revenue",
    y="Total_Pages"
)

plt.title("Browsing Activity vs Purchase")
plt.xlabel("Purchase Made")
plt.ylabel("Total Pages Visited")

plt.show()
```

Figure 5. Browsing activity versus purchase.



```
In [25]: plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="Revenue",
    y="ProductRelated"
)

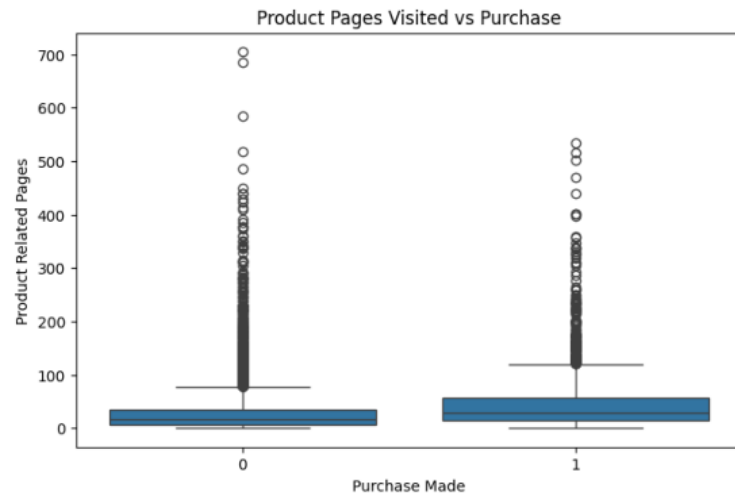
plt.title("Product Pages Visited vs Purchase")
plt.xlabel("Purchase Made")
plt.ylabel("Product Related Pages")

plt.show()
```

Figure 6. Product-related pages versus purchase.

4.6 Platform Analysis

The dataset does not contain a direct device-type variable. The notebook uses OperatingSystems as a platform-related variable. Observed conversion rates range from 0.1053 for OperatingSystem 6 to 0.2267 for OperatingSystem 8.



```
In [26]: device_conversion = (
df.groupby("OperatingSystems")["Revenue"]
    .mean()
    .sort_values(ascending=False)
)
print(device_conversion)
```

```
OperatingSystems
8    0.226667
4    0.177824
2    0.176579
5    0.166667
1    0.148686
7    0.142857
3    0.105929
6    0.105263
Name: Revenue, dtype: float64
```

```
In [27]: plt.figure(figsize=(10,5))

device_conversion.plot(kind="bar")

plt.title("Conversion Rate by Operating System")
plt.xlabel("Operating System")
plt.ylabel("Conversion Rate")

plt.show()
```

Figure 7. Conversion rate by operating system.

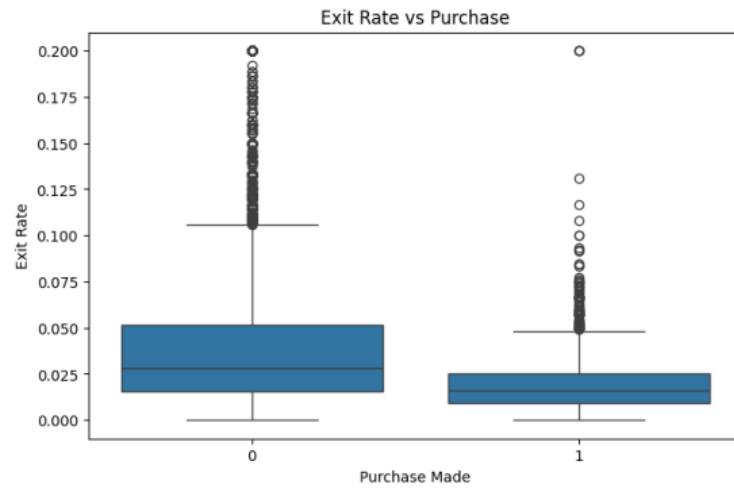
4.7 Bounce Rate, Exit Rate and Page Value

The notebook visualizes BounceRates, ExitRates and PageValues against Revenue. These variables are also included in the classification model.

```
sns.boxplot(
    data=df,
    x="Revenue",
    y="ExitRates"
)

plt.title("Exit Rate vs Purchase")
plt.xlabel("Purchase Made")
plt.ylabel("Exit Rate")

plt.show()
```



```
In [30]: plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="Revenue",
    y="PageValues"
)

plt.title("Page Value vs Purchase")
plt.xlabel("Purchase Made")
plt.ylabel("Page Value")

plt.show()
```

Figure 8. Page value versus purchase.

5. MACHINE LEARNING MODELS

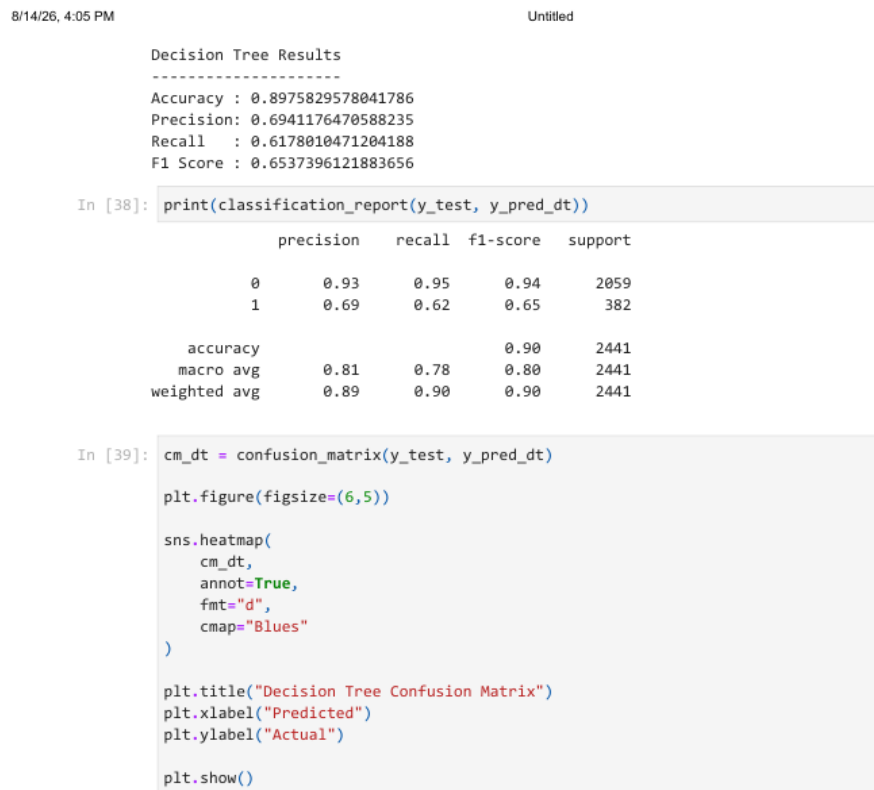
5.1 Train-Test Split

The cleaned and engineered dataset is split using an 80:20 stratified train-test split. The notebook reports 9,764 training records and 2,441 testing records.

5.2 Decision Tree Classifier

A DecisionTreeClassifier is trained with random_state=42 and max_depth=6. The preprocessing stage is part of the Pipeline so that transformation and classification are applied consistently.

Decision Tree results: Accuracy = 0.897583, Precision = 0.694118, Recall = 0.617801 and F1-score = 0.653740.



file:///C:/Users/Ashish/Downloads/Untitled.html

17/23

Figure 9. Decision Tree evaluation output and confusion-matrix visualization.

The positive-class support is 382. The corresponding confusion matrix is approximately TN=1,955, FP=104, FN=146 and TP=236.

6. SUPPORT VECTOR MACHINE (SVM)

The SVM uses an RBF kernel and the same preprocessing pipeline. The notebook reports Accuracy = 0.894306, Precision = 0.740310, Recall = 0.500000 and F1-score = 0.596875.

The SVM has higher precision than the Decision Tree, but lower recall and lower F1-score for the purchase class.

8/14/26, 4:05 PM

Untitled

```
print("Recall  :", recall_svm)
print("F1 Score :", f1_svm)
```

SVM Results

Accuracy : 0.8943056124539124

Precision: 0.7403100775193798

Recall : 0.5

F1 Score : 0.596875

```
In [42]: print(classification_report(y_test, y_pred_svm))
```

	precision	recall	f1-score	support
0	0.91	0.97	0.94	2059
1	0.74	0.50	0.60	382
accuracy			0.89	2441
macro avg	0.83	0.73	0.77	2441
weighted avg	0.89	0.89	0.89	2441

```
In [43]: cm_svm = confusion_matrix(y_test, y_pred_svm)
```

```
plt.figure(figsize=(6,5))
```

```
sns.heatmap(
    cm_svm,
    annot=True,
    fmt="d",
    cmap="Greens"
)
```

```
plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```

file:///C:/Users/Ashish/Downloads/Untitled.html

19/23

Figure 10. SVM evaluation output and confusion-matrix visualization.

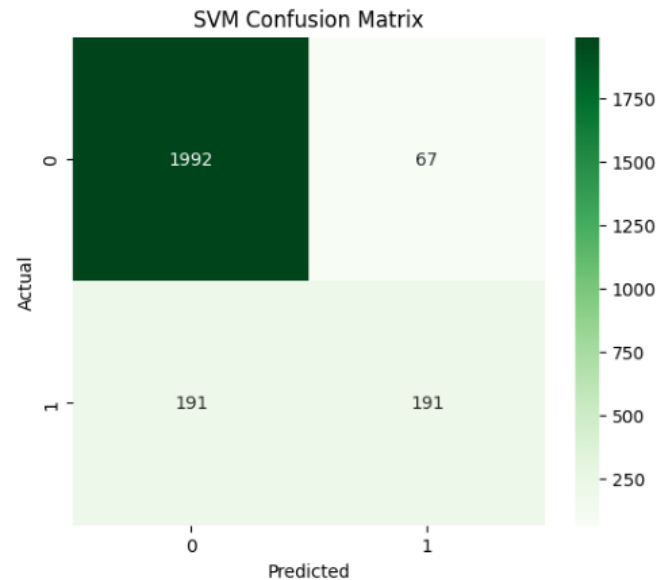
The corresponding confusion matrix is approximately TN=1,992, FP=67, FN=191 and TP=191.

7. MODEL COMPARISON

Model	Accuracy	Precision	Recall	F1-score
Decision Tree	0.897583	0.694118	0.617801	0.653740
SVM	0.894306	0.740310	0.500000	0.596875

8/14/26, 4:05 PM

Untitled



```
In [44]: results = pd.DataFrame({
    "Model": ["Decision Tree", "SVM"],
    "Accuracy": [accuracy_dt, accuracy_svm],
    "Precision": [precision_dt, precision_svm],
    "Recall": [recall_dt, recall_svm],
    "F1 Score": [f1_dt, f1_svm]
})
results
```

```
Out[44]:
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Decision Tree	0.897583	0.694118	0.617801	0.653740
1	SVM	0.894306	0.740310	0.500000	0.596875

```
In [45]: results.set_index("Model").plot(
    kind="bar",
    figsize=(10,6)
)

plt.title("Decision Tree vs SVM Performance")
plt.ylabel("Score")
plt.ylim(0,1)

plt.xticks(rotation=0)
plt.show()
```

file:///C:/Users/Ashish/Downloads/Untitled.html

20/23

Figure 11. Decision Tree versus SVM performance comparison.

The Decision Tree is selected as the preferred model for this experiment because it has higher accuracy, recall and F1-score. SVM provides higher precision but misses more actual purchasing sessions, as indicated by its lower recall.

8. FEATURE IMPORTANCE AND KEY FACTORS

The Decision Tree feature-importance output provides a model-based ranking of variables associated with purchase prediction. PageValues is the dominant feature with importance 0.736767. The next important features are BounceRates (0.072009), Month_Nov (0.043509), Total_Duration (0.028352) and ProductRelated_Duration (0.027931).

Rank	Feature	Importance
1	PageValues	0.736767
2	BounceRates	0.072009
3	Month_Nov	0.043509
4	Total_Duration	0.028352
5	ProductRelated_Duration	0.027931
6	Administrative	0.021822
7	Administrative_Duration	0.021799
8	ExitRates	0.020291
9	Total_Pages	0.008884
10	Month_Mar	0.005156

Out[46]:

	Feature	Importance
8	num_PageValues	0.736767
6	num_BounceRates	0.072009
23	cat_Month_Nov	0.043509
14	num_Total_Duration	0.028352
5	num_ProductRelated_Duration	0.027931
0	num_Administrative	0.021822
1	num_Administrative_Duration	0.021799
7	num_ExitRates	0.020291
15	num_Total_Pages	0.008884
21	cat_Month_Mar	0.005156
10	num_OperatingSystems	0.004097
25	cat_Month_Sep	0.003138
30	cat_Weekend_True	0.002164
19	cat_Month_Jul	0.001263
17	cat_Month_Dec	0.000953

```
In [47]: top_features = feature_importance.head(15)

plt.figure(figsize=(10,7))

sns.barplot(
    data=top_features,
    x="Importance",
    y="Feature"
)

plt.title("Top Factors Associated with Purchase")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.show()
```

Figure 12. Top Decision Tree factors associated with purchase prediction.

9. BUSINESS RECOMMENDATIONS

- 1. Optimize high-value pages.** PageValues is the strongest model feature, so high-value landing and product pages should receive particular attention for content quality, navigation and calls to action.
- 2. Reduce bounce and exit behavior.** BounceRates and ExitRates are among the important model variables. Improve landing-page relevance, navigation, page performance and product discovery.
- 3. Encourage useful product exploration.** Total_Duration, ProductRelated_Duration and Total_Pages are among the higher-ranked features. Product recommendations, related products, filters and comparison tools can support deeper engagement.
- 4. Evaluate traffic sources.** Conversion rates differ across TrafficType categories. Marketing resources can be evaluated using conversion rate together with traffic volume and acquisition cost.
- 5. Segment visitor experiences.** New_Visitor, Other and Returning_Visitor show different observed conversion proportions. Personalized messaging can therefore be considered for different visitor groups.
- 6. Monitor platform performance.** Operating-system conversion rates vary, so platform-specific usability can be monitored. These results should not be described as direct mobile/desktop results because device type is not present in the dataset.

10. LIMITATIONS

- The target is imbalanced: 84.37% non-purchase versus 15.63% purchase sessions after duplicate removal.
- The dataset does not provide a direct device-type field.
- The notebook uses fixed model settings (Decision Tree max_depth=6 and RBF SVM) and does not document a hyperparameter-search stage.
- Decision Tree feature importance indicates model contribution, not causal influence.

11. CONCLUSION

The project successfully implements a complete online shopping purchase-prediction workflow. The data is cleaned, explored and transformed into a machine-learning-ready form, followed by training and evaluation of Decision Tree and SVM classifiers.

The Decision Tree performs better overall in the submitted notebook, achieving 89.76% accuracy and a 65.37% F1-score, compared with 89.43% accuracy and a 59.69% F1-score for SVM. PageValues is the dominant Decision Tree feature, followed by BounceRates, Month_Nov, Total_Duration and ProductRelated_Duration.

The findings provide practical directions for an online retailer: optimize high-value pages, reduce bounce/exit behavior, encourage product engagement, evaluate traffic sources and segment visitor experiences. The model can serve as a starting point for personalized offers and conversion-focused decision making.

12. SUBMISSION REQUIREMENTS MAPPING

1. GitHub Repository: Include the executable Jupyter notebook, README, visualizations and working model code. The report can also be included in a report folder.

2. Soft-Bound Report: This report contains the problem statement, dataset, preprocessing, EDA, model methodology, evaluation, screenshots, feature importance, recommendations and conclusion.

3. Presentation: Use the report sections to prepare slides covering problem statement, dataset, preprocessing, EDA, Decision Tree, SVM, model comparison, feature importance, recommendations and conclusion.

Source note: All project-specific results in this report are based on the uploaded Jupyter Notebook PDF, including its actual dataset dimensions, cleaning results, EDA outputs, model scores and feature-importance values. ■filecite■turn0file0■L29-L34■ ■filecite■turn0file0■L148-L162■ ■filecite■turn0file0■L471-L490■ ■filecite■turn0file0■L546-L558■ ■filecite■turn0file0■L616-L631■